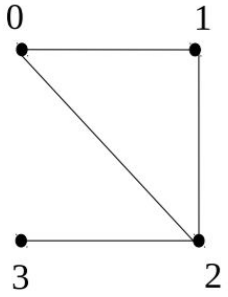
Graphs

DS 2020/2021

Graphs

- $G = (V, E)$
- V a set of vertices noduri
- E a set of edges; muchii an edge = a non-ordered pair of distinct vertices



$V = \{0, 1, 2, 3\}$ noduri

$E = \{\{0, 1\}, \{0, 2\}, \{1, 2\}, \{2, 3\}\}$ muchii

$u = \{0, 1\} = \{1, 0\}$ u is an edge, either 0,1 or 1,0

0,1 - the ends of u

u is incident with 0 and 1

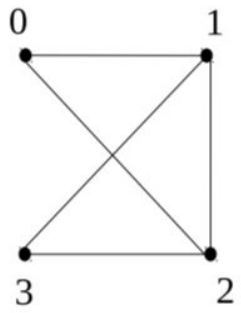
edge u is incident with vertices 0 and 1(an edge is incident with a node if it has him at his extremities)

0 and 1 are adjacent (neighbors)

2 vertices are adjacent if they have an edge in common, 0 and 1 are adjacent because they have "1" edges in common, {0,1} and {1,0}

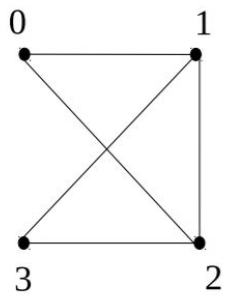
Grafuri

- Mers de la u la v (*walk*): $u = i_0, \{i_0, i_1\}, i_1, \cdots, \{i_{k-1}, i_k\}, i_k = v$
  3, {3,2}, 2, {2,0}, 0, {0,1}, 1, {1,3},3, {3,2}, 2
- parcurs (*trail*): mers în care oricare două muchii sunt distincte
- circuit = parcurs închis în care oricare două muchii intermediare sunt distincte
- drum (*path*): mers în care oricare două vârfuri sunt distincte
- ciclu (*cycle*): drum închis $i_0 = i_k$



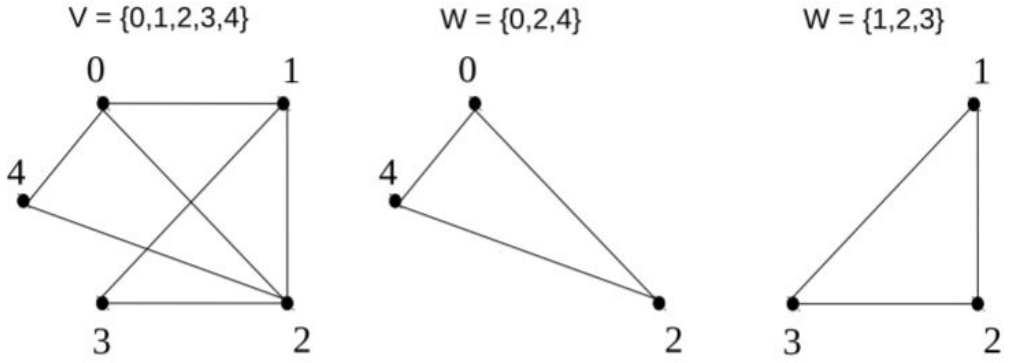
Graphs

- Walk from u to v: $u = i_0, \{i_0, i_1\}, i_1, \cdots, \{i_{k-1}, i_k\}, i_k = v$
  3, {3,2}, 2, {2,0}, 0, {0,1}, 1, {1,3},3, {3,2},2 mers
- Trail: a walk where any two edges are distinct parcurs
- Circuit = a closed trail where any two intermediate edges are distinct
- Path: a walk where any two vertices are distinct drum
- Cycle: closed path ($i_0 = i_k$) ciclu



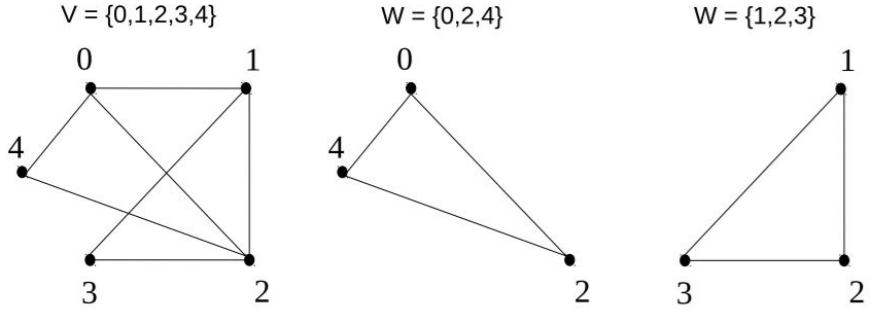
Subgraf indus

- $G = (V, E)$ – graf, W – submulțime a lui V
- Subgraf indus de W: $G'(W, E')$, unde
  $E' = \{\{i,j\} | \{i,j\} \in E$ și $i \in W, j \in W\}$



Induced subgraph

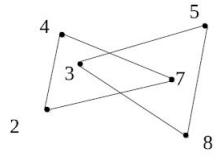
- $G = (V, E)$ – a graph, W – a subset of V
- Induced subgraph: $G'(W, E')$, where subgraf indus
  $E' = \{\{i,j\} | \{i,j\} \in E$ și $i \in W, j \in W\}$



Graphs - Connectivity

Any graph can be expressed as the disjoint union of induced, connected and maximal subgraphs (connected components).

- $\boxed{R}$ is an equivalence relation
- $i \boxed{R} j$ if and only if there is a path from i to j
- $V_1, \cdots, V_p$ equivalence classes
- $G_1, \cdots, G_p$ connected components, where $G_i = (V_i, E_i)$ a subgraph induced by V_i



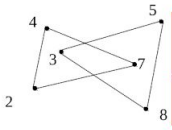
$V_1 = \{2, 4, 7\}$
$E_1 = \{\{2, 4\}, \{4, 7\}, \{2, 7\}\}$
$V_2 = \{3, 5, 8\}$
$E_2 = \{\{3, 5\}, \{5, 8\}, \{8, 3\}\}$

- connected graph = a graph with a single connected component

Grafuri - Conexitate

Orice graf poate fi exprimat ca fiind reuniunea disjunctă de subgrafuri induse, conexe și maximale (componente conexe).

- $\boxed{R}$ este relație de echivalență
- $i \boxed{R} j$ dacă și numai dacă există drum de la i la j
- $V_1, \cdots, V_p$ clasele de echivalență
- $G_1, \cdots, G_p$ componente conexe, $G_i = (V_i, E_i)$ subgraful indus de V_i



$V_1 = \{2, 4, 7\}$
$E_1 = \{\{2, 4\}, \{4, 7\}, \{2, 7\}\}$
$V_2 = \{3, 5, 8\}$
$E_2 = \{\{3, 5\}, \{5, 8\}, \{8, 3\}\}$

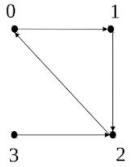
- graf conex = graf cu o singură componentă conexă

Abstract data type **Graph**

- objects:
  - graphs $G = (V, E)$, $V = \{0, 1, \cdots, n-1\}$
- operations:
  - emptyGraph()
    - input: nothing
    - output: the empty graph $(\emptyset, \emptyset)$
  - isEmptyGraph()
    - input: $G = (V, E)$,
    - output: true if $G = (\emptyset, \emptyset)$, false other way
  - insertEdge()
    - input: $G = (V, E)$, $i, j \in V$
    - output: $G = (V, E \cup \{i, j\})$
  - insertVertex()
    - input: $G = (V, E)$, $V = \{0, 1, \cdots, n-1\}$
    - output: $G = (V', E)$, $V' = \{0, 1, \cdots, n-1, n\}$

Digraph (directed graph)

- $D = (V, A)$
  - V a set of vertices
  - A a set of arcs (directed edges); an arc = an ordered pair of distinct vertices

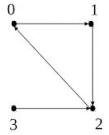


$V = \{0, 1, 2, 3\}$
$A = \{(0, 1), (2, 0), (1, 2), (3, 2)\}$
$a = (0, 1) \neq (1, 0)$

0 – the tail of a
1 – the head of a

Digraf (graf orientat)

- $D = (V, A)$
  - V mulțime de vârfuri
  - A mulțime de arce; un arc = o pereche ordonată de vârfuri distincte



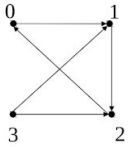
$V = \{0, 1, 2, 3\}$
$A = \{(0, 1), (2, 0), (1, 2), (3, 2)\}$
$a = (0, 1) \neq (1, 0)$

0 – sursa lui a
1 – destinația lui a

Digraph

- Walk: $i_0, (i_0, i_1), i_1, \cdots, (i_{k-1}, i_k), i_k$
  3, (3,2), 2, (2,0), 0, (0,1), 1, (1,2), 2, (2,0), 0
- Trail: a walk where any two arcs are distinct
- Circuit = a closed trail where any two intermediate arcs are distinct

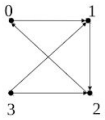
- Path: a walk where any two vertices are distinct
- Cycle: closed path ($i_0 = i_k$)



Digraf

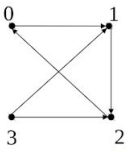
- mers (*walk*): $i_0, (i_0, i_1), i_1, \cdots, (i_{k-1}, i_k), i_k$
  3, (3,2), 2, (2,0), 0, (0,1), 1, (1,2), 2, (2,0), 0
- parcurs (*trail*): mers în care oricare două arce sunt distincte
- circuit = parcurs închis în care oricare două arce intermediare sunt distincte

- drum (*path*): mers în care oricare două vârfuri sunt distincte
- ciclu (*cycle*): drum închis $i_0 = i_k$



Digraph - Connectivity

- $i\ R\ j$ if and only if there is a path from i to j and a path from j to i
- R is an equivalence relation
- $V_1, \cdots, V_p$ the equivalence classes
- $G_1, \cdots, G_p$ strongly connected components, where $G_i = (V_i, A_i)$ the subdigraph induced by V_i



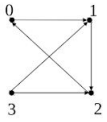
$V1 = \{0, 1, 2\}$
$A1 = \{(0, 1), (1, 2), (2, 0)\}$

$V2 = \{3\}$
$A2 = \emptyset$

- strongly connected digraph = digraph with a single strongly connected component
- connected digraph

Digraf - Conexitate

- $i\ R\ j$ dacă și numai dacă există drum de la i la j și drum de la j la i
- R este relație de echivalență
- $V_1, \cdots, V_p$ clasele de echivalență
- $G_1, \cdots, G_p$ componente tare conexe, $G_i = (V_i, A_i)$ subdigraful indus de V_i
- digraf tare conex = digraf cu o singură componentă tare conexă
- digraf conex



$V1 = \{0, 1, 2\}$
$A1 = \{(0, 1), (1, 2), (2, 0)\}$

$V2 = \{3\}$
$A2 = \emptyset$

Abstract data type **Digraph**

- objects: digraphs $D = (V, A)$
- operations:
  - emptyDigraph()
    - input: nothing
    - output: the empty digraph $(\emptyset, \emptyset)$
  - isEmptyDigraph()
    - input: $D = (V, A)$,
    - output: true if $D = (\emptyset, \emptyset)$, false other way
  - insertArc()
    - input: $D = (V, A)$, $i, j \in V$
    - output: $D = (V, A \cup (i, j))$
  - insertVertex()
    - input: $D = (V, A)$, $V = \{0, 1, \cdots, n-1\}$
    - output: $D = (V', A)$, $V' = \{0, 1, \cdots, n-1, n\}$

Content

Abstract data type Graph

Abstract data type Digraph

The implementation with adjacency matrices

The implementation with adjacency linked lists

Graph traversal algorithms (DFS, BFS)

Finding the (strongly) connected components

The implementation of digraphs with adjaceny matrices

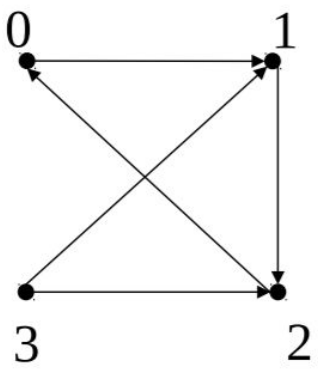
- the representation of digraphs
  - n the number of vertices
  - m the number of arcs (optional)
  - a matrix $(a[i,j] | \ 1 \leq i, \ j \leq n)$
    $a[i,j] = if \ (i,j) \in A \ then \ 1 \ else \ 0$
  - if the digraph is a graph, then $a[i,j]$ is symmetric
  - the out adjacency list of $i \subseteq$ line i
  - the in adjacency list of $i \subseteq$ column i

Implementarea cu matrici de adiacență a digrafurilor

- reprezentarea digrafurilor
  - n numărul de vârfuri
  - m numărul de arce (opțional)
  - o matrice $(a[i,j] | \ 1 \leq i, \ j \leq n)$
    $a[i,j] = if \ (i,j) \in A \ then \ 1 \ else \ 0$
  - dacă digraful reprezintă un graf, atunci $a[i,j]$ este simetrică
  - lista de adiacență exterioară a lui $i \subseteq$ linia i
  - lista de adiacență interioară a lui $i \subseteq$ coloana i

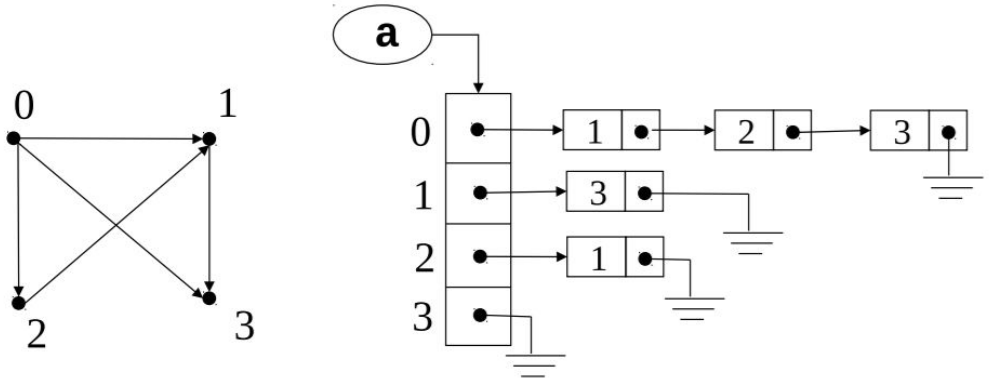
The implementation with adjacency matrices

	0	1	2	3
0	0	1	0	0
1	0	0	1	0
2	1	0	0	0
3	0	1	1	0



The implementation with adjacency lists

- reprezentarea digrafurilor cu liste de adiacență exterioară
- the representation of digraphs with (out) adjacency lists



- a vector $a[0..n-1]$ of linked lists (pointers)
- $a[i]$ is the out adjacency list corresponding to i

- un tablou $a[0..n-1]$ de liste înlănțuite (pointeri)
- $a[i]$ este lista de adiacență exterioară corespunzătoare lui i

Content

Abstract data type Graph

Abstract data type Digraph

The implementation with adjacency matrices

The implementation with adjacency linked lists

Graph traversal algorithms (DFS, BFS) already discussed

Finding the (strongly) connected components